altairsim — Printing to a real printer

2026-08-03 · 846eb61

A machine with an **88-C700 parallel printer interface** on it, and a short program that prints a banner through it. Connect the C700 to a real printer on your host, run the program, and a page comes out.

```
cd examples/printing
altairsim printer.toml
```

You land at the `altairsim>` prompt. The machine is an 8080, 32K of RAM, an 88-2SIO for the console, and the 88-C700 printer card (`lpt0`) at port 02. The card's one unit, `prn`, starts **disconnected** — because what is on the far end of the printer cable is a decision about *your host*, not about the machine. Everything below is about making that connection.

The program is `PRINT.ASM`, assembled into `PRINT.HEX`: it sends `ALTAIRSIM 8800`, a carriage return and line feed, and a **form feed** to the printer, then halts. The form feed is the byte that ejects the page — a real printer holds the sheet until something tells it the page is finished.

The one thing to understand first: when does a job print?

A printer has no "I'm done" signal — a program prints some bytes and then simply stops. altairsim reaches a real printer two ways, and they differ in *when the bytes leave*:

- **`socket:HOST:9100`** — altairsim opens a TCP connection straight to a network printer's raw port and streams the bytes. They go out **while the machine is running** (the connection is serviced by the run loop). This is the simplest route and needs no host print system at all. **Use this for a network printer.**
- **`printer:QUEUE`** — altairsim hands the bytes to your host's **print system** (CUPS on macOS/Linux) as a *job*. It buffers them and submits one job at a boundary: a few seconds idle, a form feed (`?onff`), or when you `DISCONNECT`. `DISCONNECT` submits **immediately**, so this is the one you can drive by hand from the prompt. **Use this for a USB printer, or when you don't have a network printer, or on Linux with a raw queue.**

Pick the section below that matches your host and your printer.

---

macOS

The easy way — a network printer, no setup at all

If your printer is on the network (most laser printers are), you need **nothing on the host** — no CUPS queue, no `sudo`. Find its IP (the printer's front panel under Network, or your router's device list, or `ippfind` in a terminal), confirm its raw port is open, and connect straight to it:

```
altairsim> LOAD PRINT.HEX
altairsim> CONNECT lpt0:prn socket:192.0.2.234:9100        # your printer's IP
altairsim> RUN 0100
```

A page reading `ALTAIRSIM 8800` comes out. That's it. `socket:` sends the bytes to the printer's built-in JetDirect port (9100), which every network printer with raw/AppSocket printing exposes, and the printer's own PCL/text interpreter renders them.

> **Confirmed:** this prints on a Brother MFC-L5700DW — a modern AirPrint laser — because it keeps an open port 9100 and auto-detects the raw text. If your printer is set to a fixed emulation, set it to **Auto** (the factory default).

Remember `socket:` delivers *during a* `RUN`. If you `CONNECT` and then type `OUT` bytes at the prompt, nothing leaves until the machine runs — that is expected. `RUN`ning the banner program (or booting a guest that prints) is what services the line.

The CUPS way — a USB printer, or driving it by hand

For a USB printer, or to send a job by hand without a running program, go through CUPS with `printer:`. macOS has **removed raw print queues** (`lpadmin: Raw queues are no longer supported on macOS`), and it now also warns that **drivers are deprecated** (`Printer drivers are deprecated and will stop working in a future version of CUPS`). The job you send is still raw, though, so the queue just needs a nominal generic PPD to exist:

```
# one-time: a queue named "altair" for a USB printer (use its device URI from `lpinfo -v`)
sudo lpadmin -p altair -E -v usb://Brother/MFC... \
             -m drv:///sample.drv/generic.ppd -o printer-is-shared=false
```

Then print and flush by hand:

```
altairsim> LOAD PRINT.HEX
altairsim> CONNECT lpt0:prn printer:altair
altairsim> RUN 0100
altairsim> DISCONNECT lpt0:prn      # submits the job to CUPS right now
```

`?onff` makes each form feed end a job on its own, and `?idle=N` ends one after N seconds of silence — `CONNECT lpt0:prn printer:altair?onff` prints each page as the program ejects it.

See the bytes with no paper

To watch exactly what would reach the printer — for testing, or to see the job boundaries — point a CUPS queue at a local `nc` listener instead of a printer:

```
sudo lpadmin -p altairterm -E -v socket://localhost:9100 \
             -m drv:///sample.drv/generic.ppd -o printer-is-shared=false
nc -k -l 9100                        # one terminal: raw bytes stream here as they land
```

```
altairsim> CONNECT lpt0:prn printer:altairterm     # another terminal
altairsim> LOAD PRINT.HEX
altairsim> RUN 0100
altairsim> DISCONNECT lpt0:prn
```

`ALTAIRSIM 8800` appears in the `nc` terminal. Clean text means the raw job passed straight through; PostScript boilerplate around it would mean the host filtered it (it does not, on current macOS). Remove a test queue afterwards with `sudo lpadmin -x altairterm`.

Which should I use on macOS? A network printer → `socket:` (nothing to set up). A USB printer → the CUPS `printer:` queue. macOS is walking away from raw host printing (raw queues gone, drivers deprecated), so for anything new the `socket:` route to a network printer is the one that will keep working.

Linux

Linux CUPS still supports **raw queues** properly, so it is the cleanest `printer:` host. Create a raw queue for your printer:

```
lpinfo -v                                              # find the device URI
sudo lpadmin -p altair -E -v usb://Brother/MFC... -m raw   # -m raw: no filtering
```

Then exactly as above:

```
altairsim> LOAD PRINT.HEX
altairsim> CONNECT lpt0:prn printer:altair
altairsim> RUN 0100
altairsim> DISCONNECT lpt0:prn
```

The no-paper `nc` check is the same recipe with `-m raw` in place of the generic PPD. And a network printer works with `socket:HOST:9100` on Linux just as on macOS — no queue needed.

Windows

Not yet available. The Windows print path (WinSpool, the "Generic / Text Only" driver) is designed but not built — it is a later phase of the printer work. On Windows today, a **network** printer still works through `socket:HOST:9100`, which needs no host print system. This section will grow a full walkthrough when the WinSpool backend lands.

If nothing comes out

- **A network `socket:` print produced nothing** — remember it only sends during a `RUN`. Did you `RUN 0100` after connecting? And is the printer's raw port 9100 actually open (`nc -z PRINTER-IP 9100`)?
- **The printer took the job but no page ejected** — it is holding the last page. The banner program sends a form feed (`0Ch`) to eject; a program that does not must, or use `printer:QUEUE?onff` so altairsim ends the job on the form feed and the print system ejects.
- **A `printer:` job vanished** — a failed submit is reported, not silent: it appears at the prompt after the command (the same channel a serial port uses to report trouble). A bad queue name lists the queues that exist; connect to bare `printer:` to see them.
- **Garbled output on a modern printer** — set the printer's emulation to **Auto** (or a text/PCL mode). A printer expecting only rendered pages (PDF/AirPrint) may not interpret a raw ASCII stream; a network laser with an open 9100 port almost always does.

The board itself is documented in the User Manual; the endpoint grammar (`printer:`, `socket:`, and the rest) is in the manual's serial chapter.